



Obligatorio SSTD

Ingeniería en Electrónica, Universidad ORT Uruguay

Mateo Menchaca - 251812

Mateo Silva - 282489

Docente: Lucas Cresci

Índice

1	Introducción	3
2	Parte 1	4
3	Parte 2	15
3.0.1	Justificación de diseño	16
4	Conclusión	23
5	Anexo	26
5.1	Código Scilab	26
5.1.1	IIR	26
5.1.2	FIR	43

Introducción

En el presente trabajo se aborda el diseño, la simulación y la implementación de sistemas digitales de procesamiento de señales, utilizando el entorno Scilab como herramienta principal. El proyecto se divide en dos etapas principales: la primera corresponde al desarrollo de un ecualizador digital, mientras que la segunda consiste en el diseño de un codificador y su correspondiente decodificador de señales de audio.

En la primera etapa, se diseña un ecualizador digital basado en filtros IIR, construidos mediante el método de ubicación de ceros y polos. El sistema permite ajustar de forma independiente la ganancia en distintas bandas de frecuencia, con el objetivo de modificar selectivamente el espectro de una señal de entrada. Para ello, se implementan al menos tres bandas de control, permitiendo una manipulación precisa de la respuesta en frecuencia y una verificación práctica mediante señales de audio en formato `.wav`.

La segunda etapa consiste en el diseño de un sistema codificador–decodificador que permite enmascarar una señal vocal con el fin de dificultar escuchas no autorizadas en un canal de comunicación analógico. El codificador divide la señal vocal en sub-bandas y traslada cada una a diferentes regiones de frecuencia, mientras que el decodificador realiza el proceso inverso, restituyendo las componentes a su posición original y recuperando la señal de manera aproximada. Ambos módulos se desarrollan a partir de filtros FIR de fase pseudo-lineal, garantizando una adecuada reconstrucción temporal y espectral.

Parte 1

Polos y ceros en filtros IIR

En el análisis y diseño de filtros digitales, el concepto de polos y ceros resulta fundamental para comprender el comportamiento en frecuencia y estabilidad de un sistema. Su función de transferencia en el dominio de z puede expresarse de forma general como:

$$H(z) = \frac{B(z)}{A(z)} = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2} + \dots + b_M z^{-M}}{1 + a_1 z^{-1} + a_2 z^{-2} + \dots + a_N z^{-N}}$$

Los ceros del sistema son las raíces del numerador $B(z)$, es decir, los valores de z que anulan la respuesta $H(z)$. Estos determinan las frecuencias en las que el sistema atenúa o cancela la señal. Por otro lado, los polos son las raíces del denominador $A(z)$ y representan los valores de z para los cuales la respuesta del sistema tiende a infinito.

En el caso de filtros analógicos expresados en el dominio de s , los polos y ceros son las raíces de los polinomios del numerador y denominador de la función $H(s)$. Mediante transformaciones bilineales o equivalentes, estos pueden trasladarse al plano z para obtener la versión digital del filtro.

Geométricamente, cada cero en el plano z genera una región de atenuación en la respuesta en frecuencia, mientras que cada polo refuerza la respuesta en su entorno. El equilibrio entre la ubicación de polos y ceros permite definir el tipo de filtro (pasa-bajo, pasa-banda, pasa-alto, etc.) y controlar su ganancia, pendiente y comportamiento de fase.

Durante el diseño del ecualizador digital, los polos y ceros se ubican cuidadosamente en el plano z de modo de modificar selectivamente la amplitud de ciertas bandas de frecuencia. Ajustando sus posiciones radiales (ρ) y angulares (φ), se controla tanto la ganancia como el ancho de cada banda, permitiendo lograr la respuesta deseada del sistema.

Una vez introducidos los conceptos fundamentales de polos y ceros y su relevancia en el diseño de filtros IIR, se procede a presentar el filtro inicial proporcionado por el docente, el cual constituye la base para las modificaciones requeridas en la consigna.

En la figura de la izquierda se muestra la ubicación de los polos y ceros del sistema en el plano z , representados sobre los ejes real e imaginario. Esta representación permite visualizar la influencia que ejerce cada raíz sobre la respuesta en frecuencia del filtro, así como verificar su estabilidad a partir de la posición relativa respecto al círculo unitario.

Por otra parte, en la figura del ecualizador muestra la respuesta en frecuencia correspondiente a dicho conjunto inicial de polos y ceros. Esta gráfica constituye la referencia a partir de la cual se realizaron los ajustes necesarios para obtener el ecualizador requerido. Cabe recalcar que los códigos los cuales generan las siguientes gráficas fueron proporcionadas por el docente como apoyo para el desarrollo del obligatorio.

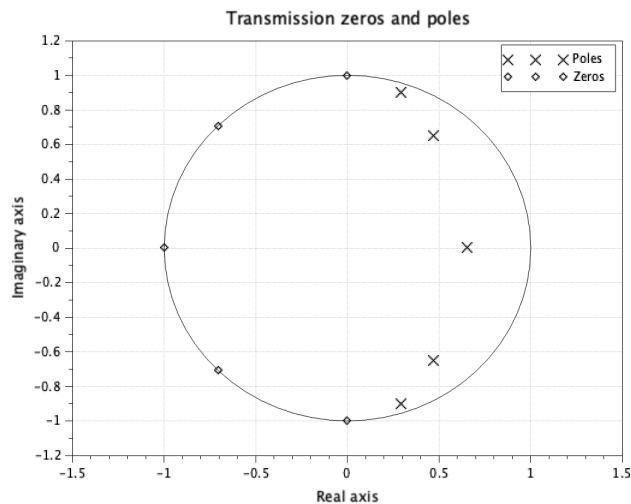


Figure 2.1: Diagrama de ceros y polos iniciales

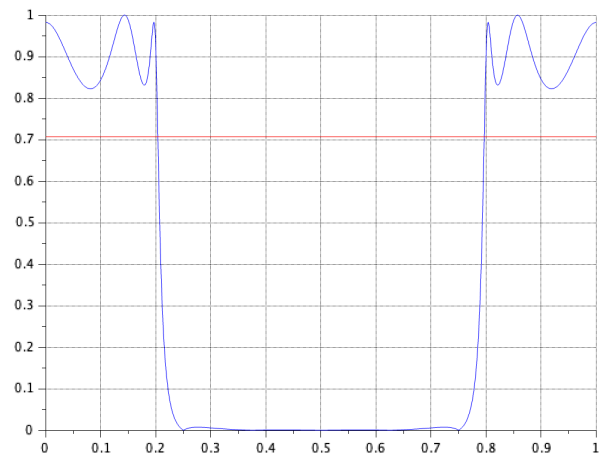


Figure 2.2: Ecualizador digital

Habiendo mencionado esto, vamos a comenzar la explicación de nuestro procedimiento

Procedimiento para conseguir los filtros deseados

Procedimiento de diseño del ecualizador

Para la primera parte del obligatorio se adoptó una estrategia modular basada en el diseño independiente de tres filtros IIR, un filtro pasa-bajo, un filtro pasa-alto y un filtro pasa-banda. Esta descomposición permite actuar selectivamente sobre distintas regiones del espectro y facilita el ajuste independiente de la ganancia en cada banda, cumpliendo con el requisito planteado en la consigna.

Los filtros IIR presentan la ventaja de permitir respuestas con pendientes pronunciadas mediante un número reducido de coeficientes, lo cual es especialmente útil para la construcción de un ecualizador digital de complejidad moderada. Al diseñar cada banda por separado, se obtiene un control más claro sobre el comportamiento espectral del sistema y se simplifica la verificación experimental de cada sección.

Como punto de partida se utilizó el código proporcionado por el docente, el cual generaba automáticamente el diagrama de polos y ceros y la respuesta en frecuencia asociada. A partir de esta herramienta se modificaron los parámetros radiales ρ y angulares φ de los polos y ceros, lo que permitió desplazar selectivamente las frecuencias de corte y ajustar las ganancias de cada banda.

La manipulación simultánea de ρ y φ resulta fundamental en este tipo de diseño: mientras que φ determina la ubicación angular de cada raíz, por lo tanto, la frecuencia central o de corte, el parámetro ρ controla el efecto de refuerzo o atenuación, así como la selectividad del filtro. De esta manera, el procedimiento permitió construir un ecualizador digital ajustable, compuesto por tres bandas claramente diferenciadas, cuya respuesta final se obtiene sumando apropiadamente la salida de los tres filtros diseñados.

Como criterio para evaluar la precisión de cada filtro se adoptó un umbral de error de 3dB, medido con respecto a la línea de referencia marcada en rojo en las gráficas. Este margen permite determinar con claridad los puntos de corte efectivos y verificar que la

respuesta en frecuencia se mantenga dentro de los niveles permitidos en la banda pasante.

A partir de este criterio, se definió la división espectral correspondiente para cada uno de los filtros diseñados. Cabe recordar que, debido a la simetría propia de la transformada en el dominio digital, el intervalo comprendido entre $\varphi = 0.5$ y $\varphi = 1$ representa el reflejo de la primera mitad del espectro. Por lo tanto, los límites de cada banda se consideran únicamente en el rango $0 \leq \varphi \leq 0.5$, entendiendo que su extensión hacia la segunda mitad del eje frecuencial es simétrica.

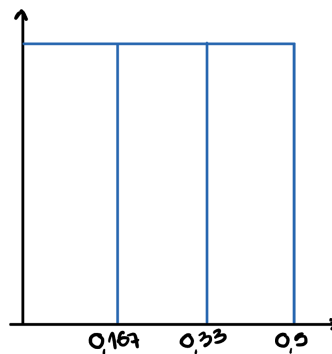


Figure 2.3: Divisiones del filtro

Viendo esta pequeña muestra de cómo serían las divisiones se intuye que la parte simétrica de 0,5 a 1 es análogo a este.

Pasa bajos

Una vez definido el procedimiento de diseño, se presentan los conjuntos de polos y ceros iniciales y finales utilizados en este trabajo. Los valores iniciales provienen directamente del código base. A partir de estos parámetros se realizaron las modificaciones necesarias, ajustando las posiciones angulares φ y radiales ρ , así como la cantidad de polos involucrados en cada banda, con el fin de adaptar la respuesta del sistema a la división en tres filtros implementada.

De esta manera, los valores finales reflejan el proceso de ajuste realizado para obtener un comportamiento acorde a lo que deseábamos llegar.

Iniciales	Finales
Polos	Polos
$\varphi_1 = 0.2, \quad \rho_1 = 0.95$	$\varphi_1 = 0.17, \quad \rho_1 = 0.9$
$\varphi_2 = 0 \quad \rho_2 = 0.65$	$\varphi_2 = 0 \quad \rho_2 = 0.7$
$\varphi_3 = 0.15, \quad \rho_3 = 0.80$	$\varphi_3 = 0.13, \quad \rho_3 = 0.8$
Ceros	Ceros
$\varphi_1 = \frac{1}{4}$	$\varphi_1 = \frac{1}{4}$
$\varphi_2 = \frac{3}{8}$	$\varphi_2 = \frac{3}{8}$
$\varphi_3 = \frac{1}{2}$	$\varphi_3 = \frac{1}{2}$

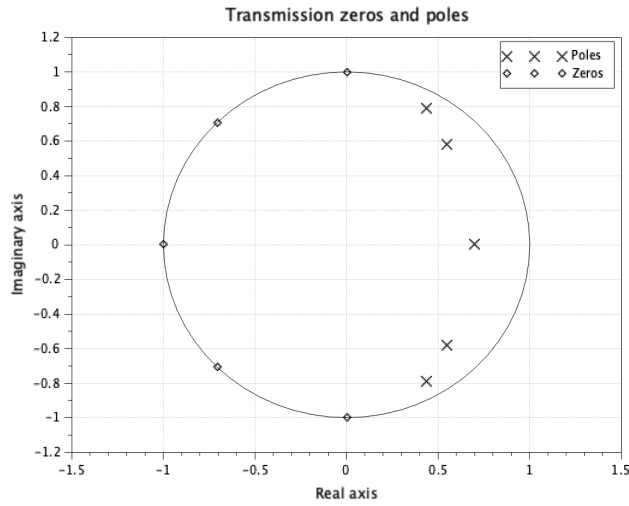


Figure 2.4: Diagrama de ceros y polos
pasa bajo

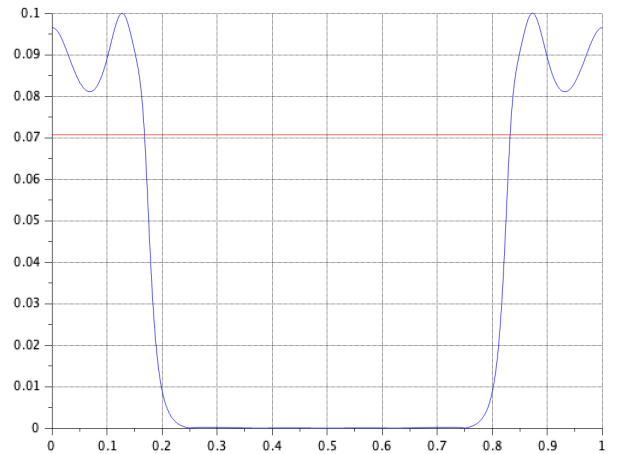


Figure 2.5: Pasa Bajo

En este gráfico se puede observar como a partir de los polos y ceros tomados se llega al filtro pasa bajo deseado, el cual corta en 0,167 con la linea roja tomada como referencia para el umbral de error.

Pasa alto

Para el diseño de este segundo filtro se comenzó utilizando valores simétricos de los polos y los ceros del pasa bajo, lo cual puede apreciarse con mayor claridad en la figura del círculo unitario. Si bien el diseño inicial empleó simetrías exactas, fue necesario realizar pequeños ajustes en los ángulos φ de los polos para lograr que la respuesta en frecuencia coincidiera con los puntos de corte definidos por la línea roja, ubicados en $\varphi = 0.167$ y $\varphi = 0.33$.

Asimismo, se modificaron ligeramente los valores radiales ρ con el fin de ajustar la selectividad del filtro.

Iniciales	Finales
Polos	Polos
$\varphi_1 = 0.67, \quad \rho_1 = 0.9$	$\varphi_1 = 0.67, \quad \rho_1 = 0.93$
$\varphi_2 = \frac{1}{2}, \quad \rho_2 = 0.7$	$\varphi_2 = \frac{1}{2}, \quad \rho_2 = 0.59$
$\varphi_3 = -0.63, \quad \rho_3 = 0.8$	$\varphi_3 = 0.63, \quad \rho_3 = 0.85$
Ceros	Ceros
$\varphi_1 = 0$	$\varphi_1 = 0$
$\varphi_2 = \frac{1}{4}$	$\varphi_2 = \frac{1}{4}$
$\varphi_3 = \frac{1}{8}$	$\varphi_3 = \frac{1}{8}$

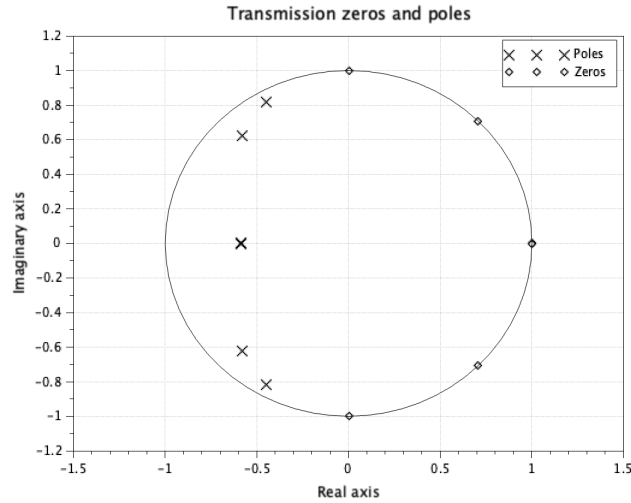


Figure 2.6: Diagrama de polos y ceros
pasa alto

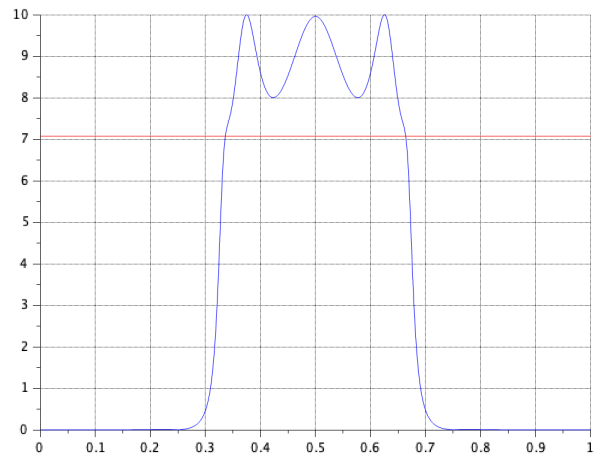


Figure 2.7: Pasa alto

Una vez visto estas representaciones, se puede observar que los polos y los ceros son simétricos al eje real y el filtro corta con la línea roja en 0,33 y 0,667 como fue mostrado al comienzo.

Pasa banda

Para esta última parte, el diseño se realizó dividiendo el círculo unitario en cuatro sectores. A partir de esta partición, se tomaron como referencia las distancias angulares utilizadas en los filtros anteriores, empleando dichas proporciones como guía para ubicar los nuevos polos y ceros. Tras efectuar los ajustes necesarios sobre estos valores, se obtuvo la configuración final del filtro.

En este caso, fue necesario incorporar dos polos adicionales respecto al diseño previo, así como un nuevo cero, con el fin de alcanzar la respuesta en frecuencia deseada. Los resultados obtenidos pueden observarse en las figuras que se presentan a continuación.

Inicial	Final
Polos	Polos
$\varphi_1 = 0.71, \quad \rho_1 = 0.93$	$\varphi_1 = 0.67, \quad \rho_1 = 0.89$
$\varphi_2 = 0.25, \quad \rho_2 = 0.90$	$\varphi_2 = 0.25, \quad \rho_2 = 0.80$
$\varphi_3 = 0.69, \quad \rho_3 = 0.85$	$\varphi_3 = 0.64, \quad \rho_3 = 0.81$
$\varphi_4 = 0.19, \quad \rho_4 = 0.85$	$\varphi_4 = 0.14, \quad \rho_4 = 0.80$
$\varphi_5 = 0.21, \quad \rho_5 = 0.93$	$\varphi_5 = 0.17, \quad \rho_5 = 0.89$
Ceros	Ceros
$\varphi_1 = 0$	$\varphi_1 = 0$
$\varphi_2 = \frac{1}{8}$	$\varphi_2 = 0.43$
$\varphi_3 = \frac{3}{8}$	$\varphi_3 = 0.05$
$\varphi_4 = \frac{1}{2}$	$\varphi_4 = 0.5$

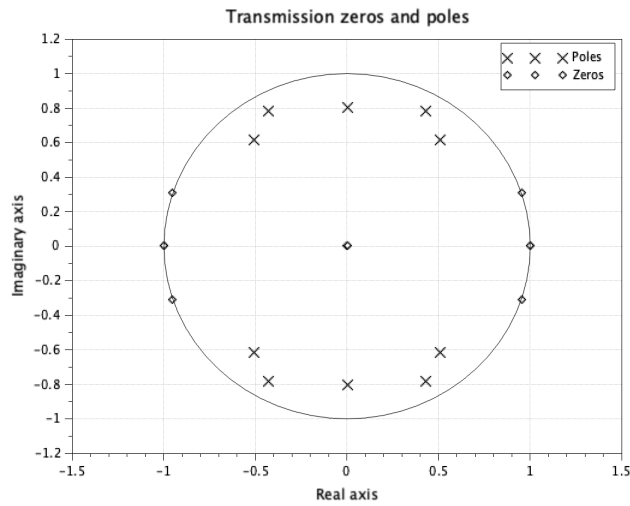


Figure 2.8: Diagrama de ceros y polos
pasa banda

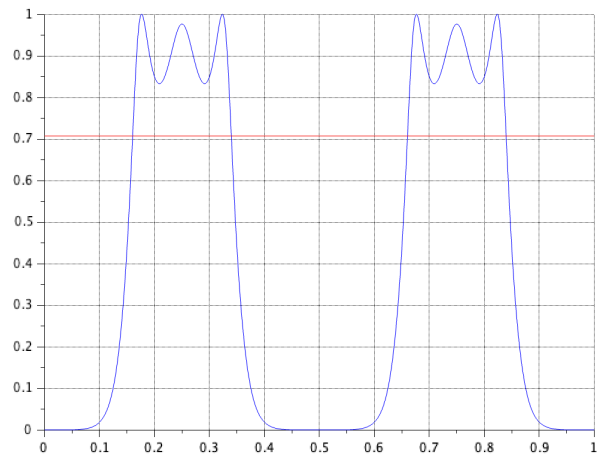


Figure 2.9: Pasa Banda

Una vez visto esto, se puede observar que los cortes con la línea roja que tomamos como limitante del umbral de error son 0,167 0,33 0,667 y 0,833 aproximadamente, los cuales son los ya mencionados en la introducción por lo que corrobora que el filtro es con las medidas ya hechas anteriormente.

Resultado final

A partir de todo lo presentado en las etapas anteriores, y utilizando otro código proporcionado por el docente, se integraron los tres filtros diseñados para conformar el ecualizador completo. Para evaluar su funcionamiento, se aplicó como entrada un archivo de audio y se incorporó un barrido en frecuencia, lo que permitió verificar visualmente si cada banda respondía según lo esperado.

En la figura siguiente se presenta la salida obtenida al aplicar el ecualizador con una ganancia de 10 en la banda de agudos, 0.1 en la banda de graves y 1 en la banda de medios. Esta configuración permite apreciar con claridad el efecto individual de cada uno de los filtros y confirmar que la respuesta final corresponde al comportamiento buscado.

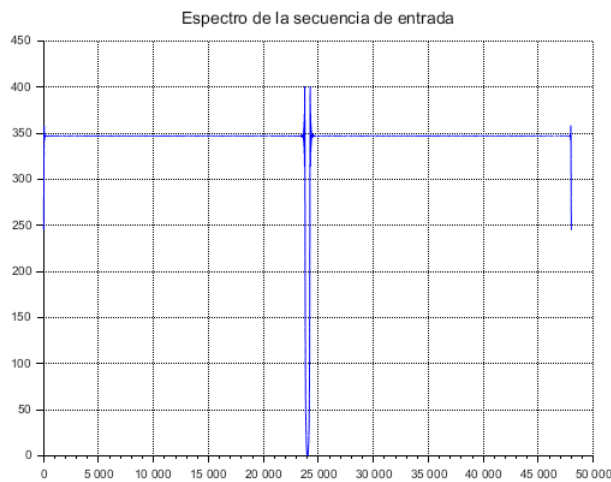


Figure 2.10: Espectro de frecuencias de la entrada

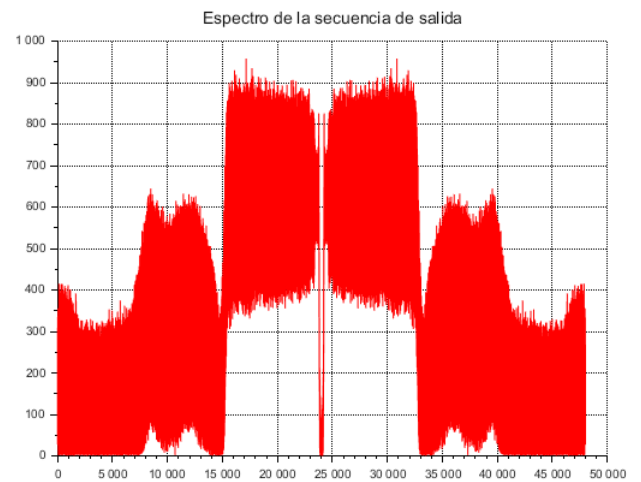


Figure 2.11: Salida filtrada con ganancia en cada banda

Parte 2

Filtro FIR

Paralelamente al diseño de filtros IIR abordado en la primera parte vamos a desarrollar otra metodología para hacer otro tipo de filtro, uno FIR. Los filtros FIR (Finite Impulse response) son incondicionalmente BIBO estables. La razón es que su respuesta al impulso es finita, lo que garantiza que la condición de estabilidad de que la respuesta sea absolutamente sumable siempre se cumpla. Además, son filtros con fase pseudolineal en la banda pasante bajo la condición suficiente de simetría/antisimetría. Esta característica garantiza que las señales atraviesen el filtro sin sufrir distorsión de fase. En contraste a los filtros IIR que, aunque son más eficientes en términos de requerimientos de cálculo, pueden volverse inestables. El principal inconveniente de los FIR es el gran número de coeficientes que requieren, lo que se traduce en mayores recursos de cálculo en comparación con un filtro IIR comparable. En definitiva los filtros FIR tienen un mayor coste en términos computacionales pero logran un resultado considerablemente mejor.

El diseño teórico de un filtro FIR se aborda mediante el método de la ventana. Este proceso parte de la determinación de una respuesta al impulso ideal que se desea, la cual es usualmente una secuencia infinita y no causal, en esta ocasión se consideran tres tipos de respuesta, el pasabajos, el pasabanda y el pasa alto. Para hacer el filtro realizable, es necesario usar un conjunto finito de coeficientes cumpliendo con la condición de fase pseudolineal. Esta selección de coeficientes se demora y se trunca por una ventana logrando que el filtro sea causal y por tanto realizable. El resultado de truncar la respuesta al impulso ideal mediante el producto con una ventana genera rizado en las transferencias ideales deseadas y un sobrepulso del rizado al fin de la banda pasante y el inicio de la banda de rechazo, y su amplitud no depende del número de coeficientes utilizado (Fenómeno de Gibbs), pero si varía según el tipo de ventana utilizada. Cada ventana define un error propio de sobrepulso, reduciéndose a costa de una necesidad de mayor

cantidad de coeficientes.

3.0.1 Justificación de diseño

En esta parte desarrollamos un mecanismo para encriptar y desencriptar un archivo de audio empleando filtros FIR. El objetivo es mezclar el contenido espectral del audio de forma tal que el mensaje resultante sea irreconocible, permitiendo luego su recuperación mediante el proceso inverso. Para ello separamos el espectro en tres bandas —baja, media y alta— y redistribuimos sus contenidos mediante modulación y filtrado selectivo.

Para implementar cada filtro definimos: el tipo de ventana, el error máximo permitido en banda (rizado), la atenuación en rechazo y el ancho de transición. Se adoptó una atenuación mínima de 50 dB en las bandas de rechazo, lo cual sugiere el uso de una ventana de Hamming al ser la más eficiente dentro de ese rango manteniendo un número acotado de coeficientes.

Ventana	error (dB)	ancho (φ) OSB	ancho (φ) J
Hamming	-53	$\frac{4}{N}$	$\frac{3.3}{N}$

La ventana de Hamming se define como

$$w(n) = \begin{cases} 0.54 - 0.46 \cos\left(\frac{2\pi n}{N}\right), & 0 \leq n \leq N, \\ 0, & \text{resto.} \end{cases}$$

En este diseño fijamos un ancho de transición extremadamente reducido, $\Delta\varphi = 0.001$, lo cual obliga a utilizar un número considerablemente mayor de coeficientes. La consecuencia directa es que las bandas pasantes se vuelven más rectangulares, con una región útil más amplia y una zona muerta entre bandas de sólo 0.001 unidades en frecuencia normalizada. Esto mejora la separación entre las tres bandas y reduce fugas espectrales entre ellas.

A continuación se detallan los criterios adoptados para cada banda:

1. **Banda baja.** Se ubica en el inicio del espectro normalizado y posee su transición

centrada en $\phi = 0.17$. Dado el valor reducido de $\Delta\varphi$, la región pasante útil se extiende prácticamente hasta dicho punto.

2. **Banda media.** Se define alrededor de la frecuencia central ϕ_0 y con un semiancho ajustado a 0.08. El módulo permanece aproximadamente unitario en la región central, y la transición hacia las bandas vecinas ocurre dentro de un intervalo de sólo $\Delta\varphi = 0.001$ respecto a sus bordes.
3. **Banda alta.** Abarca las frecuencias medibles más altas, desde $\phi = 2/6$, hasta $\phi = 1/2$, transicionando en $\Delta\varphi = 0.001$ la respuesta interseca a la banda contigua cuando $|H| = 0.5$, lo que delimita la separación efectiva con la banda media.

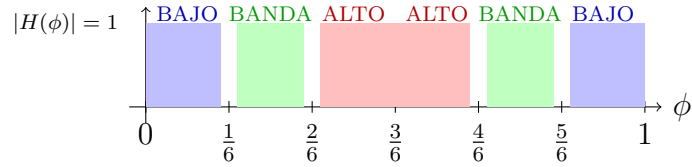


Figure 3.1: Distribución de las bandas propuesta con gaps $\Delta\varphi = 0.001$

Finalmente para hacer cada filtro usamos las siguientes expresiones analíticas de coeficientes de la respuesta al impulso para cada banda:

$$h_{Baja}[n] = (2\phi_{B,lp}) \operatorname{sinc}(\pi(2\phi_{B,lp})(n - N/2))$$

$$h_{Alta}[n] = (-1)^n (1 - 2\phi_c) \operatorname{sinc}(\pi(1 - 2\phi_c)(n - N/2))$$

$$h_{Banda}[n] = (2\phi_{B,pb}) \operatorname{sinc}(\pi(2\phi_{B,pb})(n - N/2)) 2 \cos(2\pi\phi_0(n - N/2))$$

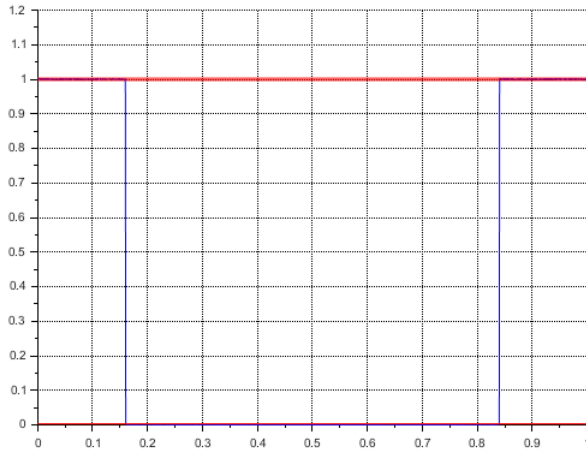


Figure 3.2: Filtro FIR pasabajos

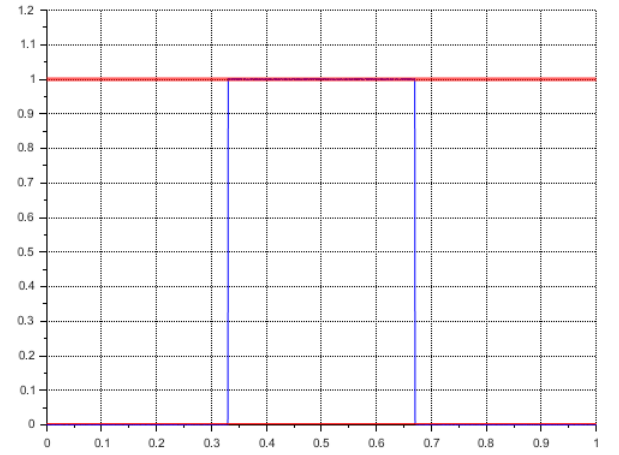


Figure 3.3: Filtro FIR pasa altos

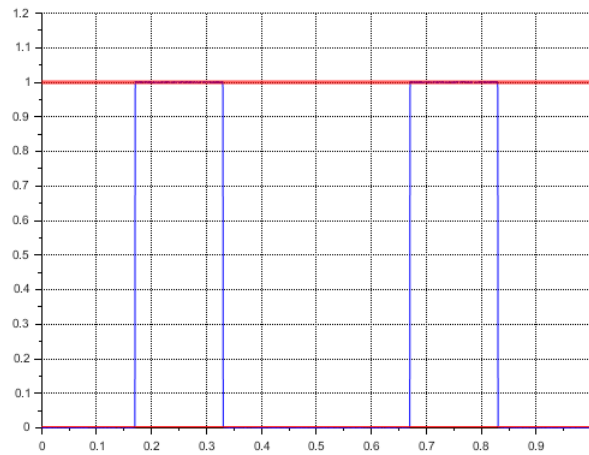


Figure 3.4: Filtro FIR pasabanda

Codificación

Para enmascarar una señal vocal comenzamos tomando el audio original y filtrándolo independientemente con cada uno de los filtros elaborados, de esta forma logramos obtener tres vectores cada uno con una banda. Luego modulamos esta banda en el espectro según el criterio definido, esto lo hacemos multiplicando por $2 x_{\text{vec}}[n] \cos(\alpha 2\pi n)$.

El criterio para mezclar las bandas define que α utilizar para cada caso, para llevar los bajos a la zona alta usamos $\alpha = 2/6$, mientras que para llevar los medio a la zona baja y los altos a la zona media usamos $\alpha = 1/6$. Paralelamente se multiplica por 2 para compensar la atenuación propia de modular y se calcula n para la entrada utilizando $n = 0 : 1 : (\text{length}(x_{\text{salida}}) - 1)$, logrando evaluar la función para cada elemento del vector de datos.

Una vez moduladas volvemos a filtrar cada banda con el filtro correspondiente a su nueva ubicación, esto nos ayuda a eliminar copias espejadas producto de la modulación y ruido introducido en el proceso, con el re filtrado volvemos a los estándares originalmente buscados.



Figure 3.5: Audio de entrada

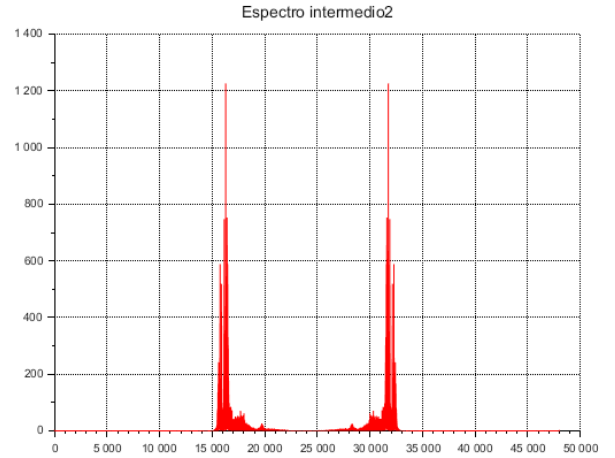


Figure 3.6: Banda baja modulada a zona alta y filtrada

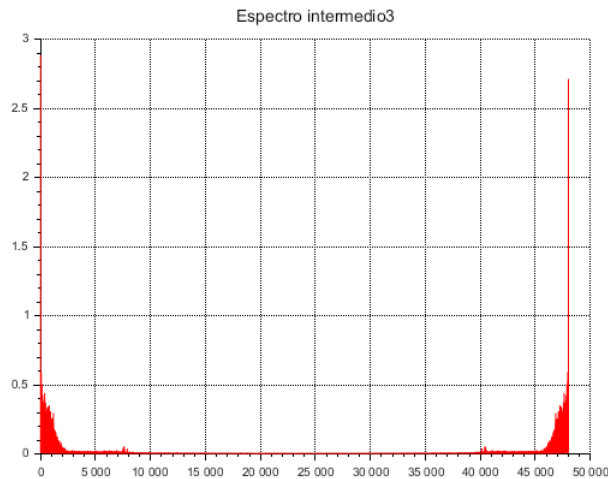


Figure 3.7: Banda media modulada a zona baja y filtrada

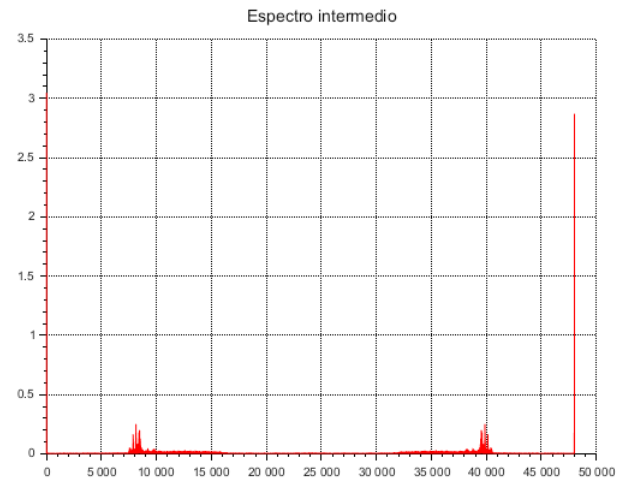


Figure 3.8: Banda alta modulada a zona media y filtrada

En este punto sumamos los tres vectores independientes para generar un archivo .wav donde deberíamos escuchar el audio codificado. Dada la selección de bandas la mayor parte del audio vocal va a parar a la zona de altos, por lo que el oído humano es incapaz de percibirlo más allá de unos leves pitidos.



Figure 3.9: Salida codificada

Decodificación

Para volver al audio original es necesario realizar el proceso inverso a la codificación, se realiza el mismo proceso pero variando como se mueven las bandas de forma de volverlas a su posición original. Para realizar correctamente esta parte utilizamos únicamente el archivo de audio "salida.wav" resultante de la etapa de codificación, además de los filtros complementarios y el orden utilizado para mezclar las bandas.

- **Banda baja:** se aísla la porción alta del espectro mediante un filtro pasa altos, se demodula multiplicando por $2 \cos\left(\frac{2}{6} 2\pi n\right)$ para desplazarla a bajas frecuencias y se aplica finalmente un filtro pasabajos para suprimir las réplicas.
- **Banda media:** se extrae la región baja del espectro con un filtro pasabajos, se demodula con $2 \cos\left(\frac{1}{6} 2\pi n\right)$ para devolverla a su posición central y se utiliza un filtro pasabanda para limpiar las copias espejo.
- **Banda alta:** se selecciona la zona media mediante un filtro pasabanda, se demodula con $2 \cos\left(\frac{1}{6} 2\pi n\right)$ para restituirla a altas frecuencias y se aplica un filtro pasaaltos para eliminar componentes no deseadas.

Se suman las tres salidas y se genera un nuevo archivo .wav para comparar con la entrada original.

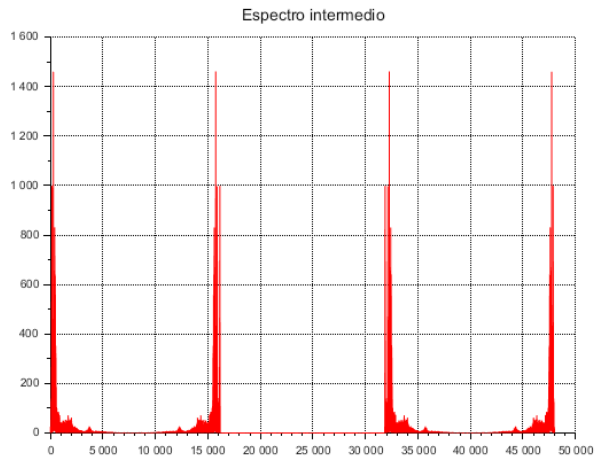


Figure 3.10: Filtrado de altos y modulación

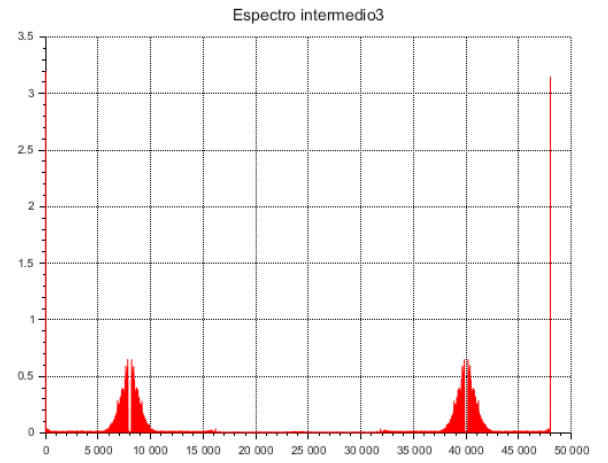


Figure 3.11: Filtrado de bajos y modulación

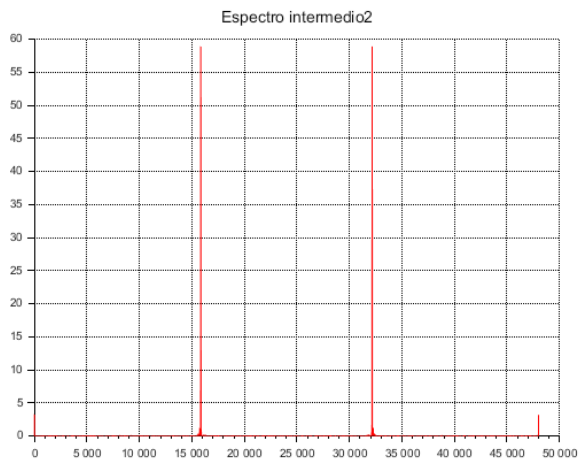


Figure 3.12: Filtrado de medios y modulación



Figure 3.13: Filtrados y suma

Conclusión

Conclusión

La primera parte del trabajo permitió implementar un ecualizador digital mediante filtros IIR. Tal como se observa en las respuestas en frecuencia obtenidas, el filtro cumple adecuadamente con las ganancias deseadas en cada banda; sin embargo, la estructura propia de los IIR introduce una distorsión inherente asociada a su fase no lineal. Esta característica hace que, si bien la ecualización es efectiva, la señal procesada presente una ligera deformación inevitable y propia del tipo de filtro utilizado.

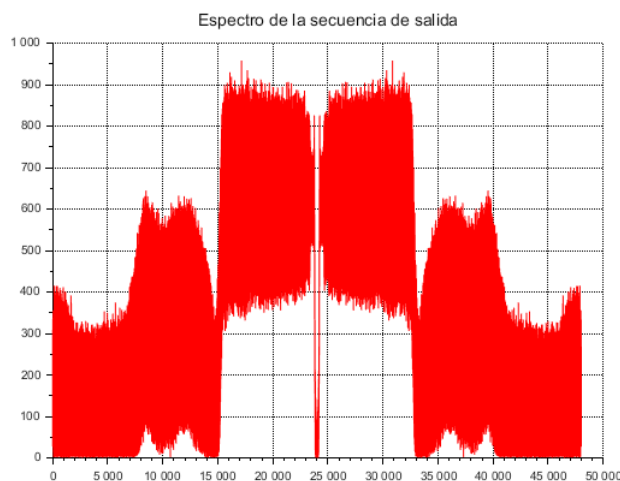


Figure 4.1: Equalización aplicada a bar-
rido lineal

En contraste, en la segunda parte se desarrolló un sistema de codificación y decodificación de audio basado en filtros FIR de fase pseudolineal. Las figuras correspondientes muestran que la separación espectral entre bandas, su modulación y el posterior reprocesamiento producen un código altamente efectivo: la señal codificada resulta completamente ininteligible para el oyente, cumpliendo así con el objetivo de enmascaramiento.

La decodificación, a su vez, demuestra la elevada precisión del método FIR. Al comparar el espectro de la señal original con el de la señal recuperada, se observa una correspondencia prácticamente exacta en las tres bandas procesadas. Sólo aparece una filtración de muy baja amplitud en regiones alejadas del contenido vocal, atribuible a las transiciones finitas de los filtros, pero su magnitud es despreciable y no afecta en la calidad del audio. De hecho, la señal decodificada se escucha con excelente fidelidad y no presenta diferencias audibles respecto del archivo original.

En síntesis, mientras el filtro IIR cumple su función de ecualización a costa de la distorsión característica de su fase, el sistema FIR logra un proceso de codificación–decodificación altamente robusto, preciso y sin distorsiones perceptibles, confirmando la superioridad de esta técnica cuando se requiere preservar la forma temporal del audio.



Figure 4.2: Entrada original



Figure 4.3: Salida Codificada



Figure 4.4: Salida Decodificada

Anexo

5.1 Código Scilab

5.1.1 IIR

Filtro IIR Pasabanda

```
1 // archivo
    verificacion_disenio_filtro_IIR_por_ubicacion_ceros_y_polos.sce
2 //
3 // Verificación de un filtro IIR diseñado por ubicación de polos y
    ceros
4 // Filtro PASABANDA
5 //-----
6
7 clear; close(winsid());
8
9 //-----
10 // Definición de ceros del filtro
11 //-----
12
13 phic1 = 0;
14 v_bi1 = [1, -2*cos(2*%pi*phic1), 1];
15
16 phic2 = 0.45;
17 v_bi2 = [1, -2*cos(2*%pi*phic2), 1];
18
19 phic3 = 0.05;
20 v_bi3 = [1, -2*cos(2*%pi*phic3), 1];
21
```

```

22 phic4 = 1/2;
23 v_bi4 = [1, -2*cos(2*%pi*phic4), 1];
24
25 // polinomio numerador total (producto de biquad)
26 v_bit = convol(convol(convol(v_bi1, v_bi2), v_bi3), v_bi4);
27
28
29 //-----
30 // Definición de polos del filtro
31 //-----
32
33 rop1 = 0.89; phip1 = 0.67;
34 v_ai1 = [1, -2*rop1*cos(2*%pi*phip1), rop1^2];
35
36 rop2 = 0.80; phip2 = 0.25;
37 v_ai2 = [1, -2*rop2*cos(2*%pi*phip2), rop2^2];
38
39 rop3 = 0.80; phip3 = 0.64;
40 v_ai3 = [1, -2*rop3*cos(2*%pi*phip3), rop3^2];
41
42 rop4 = 0.80; phip4 = 0.14;
43 v_ai4 = [1, -2*rop4*cos(2*%pi*phip4), rop4^2];
44
45 rop5 = 0.89; phip5 = 0.17;
46 v_ai5 = [1, -2*rop5*cos(2*%pi*phip5), rop5^2];
47
48 // polinomio denominador total
49 v_ait = convol(convol(convol(convol(v_ai1, v_ai2), v_ai3), v_ai4),
50               v_ai5);
51
52 //-----

```

```

53 // Preparación de la función H(z)
54 //-----
55
56 // numerador y denominador en forma polinómica en z
57 v_bi = v_bit;
58 v_ai = v_ait;
59
60 num_z1 = poly(v_bi($:-1:1), "z", "c");
61 den_z1 = poly(v_ai($:-1:1), "z", "c");
62
63 // igualar grados entre numerador y denominador
64 N_D = length(v_bi) - length(v_ai);
65 p_N_D = poly([zeros(1, abs(N_D)), 1], "z", "c");
66
67 if (N_D > 0) then
68     den_z1 = den_z1 * p_N_D;
69 elseif (N_D < 0) then
70     num_z1 = num_z1 * p_N_D;
71 end
72
73
74 //-----
75 // Evaluación sobre la circunferencia unidad
76 //-----
77
78 L = 1000;
79 v_phi = (1/L) * (0:L);
80 v_z = exp(%i * 2 * %pi * v_phi);
81
82 // evaluación de H(z)
83 v_h_phi = freq(num_z1, den_z1, v_z);
84

```

```

85
86 //-----
87 // Normalización de ganancia
88 //-----
89
90 G1 = 1;
91 gaux = max(abs(v_h_phi));
92 num_z1 = G1 * (1/gaux) * num_z1;
93
94 // reevaluamos
95 v_h_phi = freq(num_z1, den_z1, v_z);
96
97
98 //-----
99 // Gráfico de la magnitud
100 //-----
101
102 scf(1);
103 clf(); xgrid();
104 plot2d(v_phi, abs(v_h_phi), style=2);
105 plot2d(v_phi, G1*(1/sqrt(2))*ones(v_phi), style=5); // -3dB
106
107 //-----
108 // Constelación de polos y ceros
109 //-----
110
111 scf(3);
112 clf(); xgrid();
113 plzr(num_z1 / den_z1);
114
115 //-----
116 // Guardado del filtro

```

```

117 //-----
118
119 save("PasaBandaFIN", "num_z1", "den_z1");

```

Filtro IIR Pasabajos

```

1 // archivo
    verificacion_disenio_filtro_IIR_por_ubicacion_ceros_y_polos.sce
2 //
3 // Verificación de un filtro IIR diseñado por ubicación de ceros y
    polos
4 // Filtro PASABAJO
5 //-----
6
7 clear; close(winsid());
8
9 //-----
10 // Definición de ceros
11 //-----
12
13 phic1 = 1/4;
14 v_bi1 = [1, -2*cos(2*%pi*phic1), 1];
15
16 phic2 = 3/8;
17 v_bi2 = [1, -2*cos(2*%pi*phic2), 1];
18
19 v_bi3 = [1, 1];
20
21 // numerador total
22 v_bit = convol(convol(v_bi1, v_bi2), v_bi3);
23
24
25 //-----

```

```

26 // Definición de polos
27 //-----
28
29 rop1 = 0.90; phip1 = 0.17;
30 v_ai1 = [1, -2*rop1*cos(2*%pi*phip1), rop1^2];
31
32 rop2 = 0.70;
33 v_ai2 = [1, -rop2];
34
35 rop3 = 0.80; phip3 = 0.13;
36 v_ai3 = [1, -2*rop3*cos(2*%pi*phip3), rop3^2];
37
38 // denominador total
39 v_ait = convol(convol(v_ai1, v_ai2), v_ai3);
40
41
42 //-----
43 // Preparación de H(z)
44 //-----
45
46 v_bi = v_bit;
47 v_ai = v_ait;
48
49 num_z2 = poly(v_bi($:-1:1), "z", "c");
50 den_z2 = poly(v_ai($:-1:1), "z", "c");
51
52 // igualación de grados
53 N_D = length(v_bi) - length(v_ai);
54 p_N_D = poly([zeros(1, abs(N_D)), 1], "z", "c");
55
56 if (N_D > 0) then
57     den_z2 = den_z2 * p_N_D;

```

```

58 elseif (N_D < 0) then
59     num_z2 = num_z2 * p_N_D;
60 end
61
62
63 //-----
64 // Evaluación sobre la circunferencia unidad
65 //-----
66
67 L = 1000;
68 v_phi = (1/L) * (0:L);
69 v_z = exp(%i * 2 * %pi * v_phi);
70
71 v_h_phi = freq(num_z2, den_z2, v_z);
72
73
74 //-----
75 // Normalización de ganancia
76 //-----
77
78 G1 = 0.1;
79 gaux = max(abs(v_h_phi));
80
81 num_z2 = G1 * (1/gaux) * num_z2;
82
83 // reevaluación
84 v_h_phi = freq(num_z2, den_z2, v_z);
85
86
87 //-----
88 // Gráfica del módulo de la transferencia
89 //-----

```



```

90
91 scf(1);
92 clf(); xgrid();
93 plot2d(v_phi, abs(v_h_phi), style=2);
94 plot2d(v_phi, G1*(1/sqrt(2))*ones(v_phi), style=5); // -3 dB
95
96
97 //-----
98 // Constelación polos-ceros
99 //-----
100
101 scf(3);
102 clf(); xgrid();
103 plzr(num_z2 / den_z2);
104
105 //-----
106 // Guardado del filtro
107 //-----
108
109 save("Pasabajos", "num_z2", "den_z2");

```

Filtro IIR Pasa Altos

```

1 // archivo
   verificacion_diseño_filtro_IIR_por_ubicación_ceros_y_polos.sce
2 //
3 // Verificación de un filtro IIR diseñado por ubicación de ceros y
   polos
4 // Filtro PASA ALTOS
5 //-----
6
7 clear; close(winsid());
8

```

```

9  //-----
10 // Definición de ceros
11 //-----
12
13 phic1 = 0;
14 v_bi1 = [1, -2*cos(2*%pi*phic1), 1];
15
16 phic2 = 1/4;
17 v_bi2 = [1, -2*cos(2*%pi*phic2), 1];
18
19 phic3 = 1/8;
20 v_bi3 = [1, -2*cos(2*%pi*phic3), 1];
21
22 phic4 = 1/2;
23 v_bi4 = [1, -2*cos(2*%pi*phic4), 1];
24
25 // numerador total (excepto último por error original)
26 v_bit = convol(convol(v_bi1, v_bi2), v_bi3);
27
28
29 //-----
30 // Definición de polos
31 //-----
32
33 rop1 = 0.85; phip1 = 0.63;
34 v_ai1 = [1, -2*rop1*cos(2*%pi*phip1), rop1^2];
35
36 rop2 = 0.59; phip2 = 0.50;
37 v_ai2 = [1, -2*rop2*cos(2*%pi*phip2), rop2^2];
38
39 rop3 = 0.93; phip3 = 0.67;
40 v_ai3 = [1, -2*rop3*cos(2*%pi*phip3), rop3^2];

```

```

41
42 // denominador total
43 v_ait = convol(convol(v_ai1, v_ai2), v_ai3);
44
45
46 //-----
47 // Preparación de H(z)
48 //-----
49
50 v_bi = v_bit;
51 v_ai = v_ait;
52
53 num_z = poly(v_bi($:-1:1), "z", "c");
54 den_z = poly(v_ai($:-1:1), "z", "c");
55
56 // igualación de grados
57 N_D = length(v_bi) - length(v_ai);
58 p_N_D = poly([zeros(1, abs(N_D)), 1], "z", "c");
59
60 if (N_D > 0) then
61     den_z = den_z * p_N_D;
62 elseif (N_D < 0) then
63     num_z = num_z * p_N_D;
64 end
65
66
67 //-----
68 // Evaluación sobre circunferencia unidad
69 //-----
70
71 L = 1000;
72 v_phi = (1/L) * (0:L);

```

```

73 v_z = exp(%i * 2 * %pi * v_phi);
74
75 v_h_phi = freq(num_z, den_z, v_z);
76
77
78 //-----
79 // Normalización de ganancia
80 //-----
81
82 G1 = 10;
83 gaux = max(abs(v_h_phi));
84 num_z = G1 * (1/gaux) * num_z;
85
86 // Reevaluación
87 v_h_phi = freq(num_z, den_z, v_z);
88
89
90 //-----
91 // Gráfica del módulo de la transferencia
92 //-----
93
94 scf(1);
95 clf(); xgrid();
96 plot2d(v_phi, abs(v_h_phi), style=2);
97 plot2d(v_phi, G1*(1/sqrt(2))*ones(v_phi), style=5); // -3 dB
98
99
100 //-----
101 // Constelación polos-ceros
102 //-----
103
104 scf(3);

```

```

105 clf(); xgrid();
106 plzr(num_z / den_z);
107
108 //-----
109 // Guardado del filtro
110 //-----
111
112 save("PasaAltosFIN", "num_z", "den_z");

```

Filtrado con los filtros IIR diseñados

```

1  //////////////////////////////////////
2  // archivo leer_filtrar_con_filtro_IIR_y_guardar_audio.sce
3  //
4  // Filtrado de una secuencia .wav usando filtros IIR previamente
   construidos.
5  // Los filtros deben estar guardados como numerador y denominador (
   num_z, den_z)
6  // con exponentes positivos de z.
7  //////////////////////////////////////
8
9
10 //////////////////////////////////////
11 // Lectura de la secuencia de entrada
12 //////////////////////////////////////
13
14 [x_aux, Fm, bits] = wavread("Barrido2.wav");
15
16 size(x_aux)

```

```

17 Fm
18 bits
19
20 x_entrada = x_aux(1,:);
21 size(x_entrada)
22
23
24 //////////////////////////////////////
25 // Filtro PASA ALTOS
26 //////////////////////////////////////
27
28 load("PasaAltosFIN");
29
30 x_salida1 = rtitr(num_z, den_z, x_entrada);
31
32
33 //////////////////////////////////////
34 // Filtro PASA BANDA
35 //////////////////////////////////////
36
37 load("PasaBandaFIN");
38
39 x_salida2 = rtitr(num_z1, den_z1, x_entrada);
40
41
42 //////////////////////////////////////
43 // Filtro PASA BAJOS

```

44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

```
////////////////////////////////////  
  
  
load("Pasabajos");  
  
x_salida3 = rtitr(num_z2, den_z2, x_entrada);  
  
////////////////////////////////////  
  
// Suma de las tres salidas  
////////////////////////////////////  
  
  
x_salida = x_salida1 + x_salida2 + x_salida3;  
  
  
////////////////////////////////////  
  
// Guardado del archivo filtrado  
////////////////////////////////////  
  
  
wavwrite(x_salida, Fm, "salida.wav");  
  
  
////////////////////////////////////
```

Visualización del espectro

1

```
////////////////////////////////////
```

```

2 // archivo visualizar_el_espectro_de_secuencias_de_entrada_y_salida.
   sce
3 //
4 // Lectura de archivos .wav y visualización del espectro mediante la
   TDF.
5 // Se completa la secuencia con ceros y se grafica  $|X(\phi)|$ .
6 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
7
8
9 clear;
10 close(winsid());
11
12 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
13 // Lectura de la secuencia de entrada
14 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
15
16 [x_aux, Fm, bits] = wavread("Barrido2.wav");
17
18 size(x_aux)
19 Fm
20 bits
21
22 // Selección de una única pista
23 x_entrada = x_aux(1,:);
24
25
26 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
27 // Preparación para la FFT (zero padding)

```



```

28 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
29
30 K = 10;
31 x_n = [x_entrada, zeros(1, (K-1)*length(x_entrada))];
32
33 L = 2^floor(log2(length(x_n)));
34 x_n = x_n(1:L);
35
36 // TFTD
37 x_n_phi = fft(x_n);
38
39 // Vector de frecuencia adimensional
40 v_phi = (1/L) * (0:L-1);
41
42
43 //-----
44 // Gráfico del espectro de entrada
45 //-----
46
47 scf(0); xgrid();
48 plot2d(Fm*v_phi, abs(x_n_phi), style=2);
49 xtitle("Espectro de la secuencia de entrada");
50
51
52 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
53 // Lectura de la secuencia filtrada
54 ///////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
55
56 [x_aux, Fm, bits] = wavread("salida.wav");

```

```

57
58 size(x_aux)
59 Fm
60 bits
61
62 x_salida = x_aux(1,:);
63
64
65 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
66 // Preparación para la FFT (zero padding)
67 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
68
69 K = 10;
70 x_n = [x_salida, zeros(1, (K-1)*length(x_salida))];
71
72 L = 2^floor(log2(length(x_n)));
73 x_n = x_n(1:L);
74
75 x_n_phi = fft(x_n);
76 v_phi = (1/L) * (0:L-1);
77
78
79 //-----
80 // Gráfico del espectro de salida
81 //-----
82
83 scf(1); xgrid();
84 plot2d(Fm*v_phi, abs(x_n_phi), style=5);
85 xtitle("Espectro de la secuencia de salida");

```

5.1.2 FIR

Filtro FIR Pasabanda

```
1 // archivo verificacion_diseño_filtro_FIR.sce
2
3 clear; close(winsid());
4
5 // ===== Parametros del diseño =====
6
7 Fm = 1;
8
9 // caso pasabanda
10 phi_Bpb = 0.08 / Fm;
11 phi_0    = 0.25 / Fm;
12
13 // ancho de transición
14 Delta_phi = 0.001 / Fm;
15
16 // rizado permitido
17 delta = 10^(-50/20);
18
19 // número de coeficientes (ventanas Hann/Hamming)
20 alpha = 4;
21 N = round(alpha / Delta_phi);
22
23 // ===== Kaiser (opcional) =====
24
25 A = -20*log10(delta);
26 if (A > 50) then
27     Beta = 0.1102*(A-8.7);
28 elseif (A > 21) then
29     Beta = 0.5842*((A-21)^(0.4)) + 0.07886*(A-21);
```

```

30 else
31     Beta = 0;
32 end
33
34 Nk = round((A-8)/(2.285*(2*pi*Delta_phi)));
35 // N = Nk;    // descomentar si se usa Kaiser
36
37 // ===== Ventana e indices =====
38
39 vn = 0:1:N;
40 w = window("hm", N+1);
41
42 // ===== Respuesta ideal pasabanda
43     =====
44 hbpi = (2*phi_Bpb) .* sinc(%pi*(2*phi_Bpb)*(vn - N/2)) ...
45     .* (2*cos(2*pi*phi_0*(vn - N/2)));
46
47 vhn = hbpi .* w;
48
49 // ===== Respuesta en frecuencia
50     =====
51
52 L = 2^16;
53 vphi = (1/L)*(0:1:L-1);
54
55 vhtecho = fft([vhn, zeros(1, L-length(vhn))]);
56
57 scf(0); xgrid();
58 plot2d(Fm*vphi, abs(vhtecho), style=2);
59 plot2d(Fm*vphi, (1+delta)*ones(vphi), style=5);
60 plot2d(Fm*vphi, (1-delta)*ones(vphi), style=5);
61 plot2d(Fm*vphi, delta*ones(vphi), style=5);

```

```

60
61 // guardar filtro
62 save("PasaBandaFIR", "vhn");

```

Filtro FIR Pasabajos

```

1 // diseño de un filtro FIR pasabajos mediante el método de la ventana
2
3 clear; close(winsid());
4
5 // -----
6 // parámetros del diseño
7 // -----
8
9 Fm = 1;
10
11 // frecuencia de corte
12 phi_Blz = 0.17 / Fm;
13
14 // ancho de transición
15 Delta_phi = 0.001 / Fm;
16
17 // rizado máximo permitido
18 delta = 10^(-50/20);
19
20 // número de coeficientes (Hann/Hamming)
21 alpha = 4;
22 N = round(alpha / Delta_phi);
23
24 // -----
25 // ventana de Kaiser (opcional)
26 // -----
27

```

```

28 A = -20*log10(delta);
29
30 if (A > 50) then
31     Beta = 0.1102*(A-8.7);
32 elseif (A > 21) then
33     Beta = 0.5842*((A-21)^(0.4)) + 0.07886*(A-21);
34 else
35     Beta = 0;
36 end
37
38 Nk = round((A-8)/(2.285*(2*pi*Delta_phi)));
39 // N = Nk;    // descomentar si se usa Kaiser
40
41 // -----
42 // indices y ventana
43 // -----
44
45 vn = 0:1:N;
46 w = window("hm", N+1);    // ventana Hamming
47
48 // -----
49 // respuesta ideal del pasabajos
50 // -----
51
52 hlpi = (2*phi_Blp) .* sinc(pi*(2*phi_Blp)*(vn - N/2));
53
54 // aplicar ventana
55 vhn = hlpi .* w;
56
57 // -----
58 // respuesta en frecuencia
59 // -----

```

```

60
61 L = 2^16;
62 vphi = (1/L)*(0:1:L-1);
63 vhtecho = fft([vhn, zeros(1, L-length(vhn))]);
64
65 scf(0); xgrid();
66 plot2d(Fm*vphi, abs(vhtecho), style=2);
67 plot2d(Fm*vphi, (1+delta)*ones(vphi), style=5);
68 plot2d(Fm*vphi, (1-delta)*ones(vphi), style=5);
69 plot2d(Fm*vphi, delta*ones(vphi), style=5);
70
71 vhn2 = vhn;
72
73 // guardar filtro
74 save("PasaBajosFIR", "vhn2");

```

Filtro FIR Pasaaltos

```

1 // diseño de un filtro FIR pasaaltos mediante el método de la ventana
2
3 clear; close(winsid());
4
5 // -----
6 // parámetros del diseño
7 // -----
8
9 Fm = 1;
10
11 // frecuencia de corte
12 phi_c = 0.33 / Fm;
13
14 // ancho de transición
15 Delta_phi = 0.001 / Fm;

```

```

16
17 // rizado permitido
18 delta = 10^(-50/20);
19
20 // número de coeficientes (Hamming/Hann)
21 alpha = 4;
22 N = round(alpha / Delta_phi);
23
24 // -----
25 // ventana de Kaiser (opcional)
26 // -----
27
28 A = -20*log10(delta);
29
30 if (A > 50) then
31     Beta = 0.1102*(A-8.7);
32 elseif (A > 21) then
33     Beta = 0.5842*((A-21)^(0.4)) + 0.07886*(A-21);
34 else
35     Beta = 0;
36 end
37
38 // Nk = round((A-8)/(2.285*(2*%pi*Delta_phi)));
39 // N = Nk;    // descomentar si se usa Kaiser
40
41 // -----
42 // índices y ventana
43 // -----
44
45 vn = 0:1:N;
46 w = window("hm", N+1);    // ventana Hamming
47

```



```

48 // -----
49 // respuesta ideal del pasaaltos
50 // -----
51
52 hhpi = ((-1).^vn) .* ((1 - 2*phi_c) ...
53      .* sinc(%pi*(1 - 2*phi_c)*(vn - N/2)));
54
55 // aplicar ventana
56 vhn1 = hhpi .* w;
57
58 // -----
59 // respuesta en frecuencia
60 // -----
61
62 L = 2^16;
63 vphi = (1/L)*(0:1:L-1);
64 vhtecho = fft([vhn1, zeros(1, L-length(vhn1))]);
65
66 scf(0); xgrid();
67 plot2d(Fm*vphi, abs(vhtecho), style=2);
68 plot2d(Fm*vphi, (1+delta)*ones(vphi), style=5);
69 plot2d(Fm*vphi, (1-delta)*ones(vphi), style=5);
70 plot2d(Fm*vphi, delta*ones(vphi), style=5);
71
72 // guardar filtro
73 save("PasaAlto", "vhn1");

```

Codificación del audio

```

1 // archivo leer_filtrar_con_filtro_FIR_y_guardar_audio.sce
2 //
3 // codificación del audio mediante filtros FIR y modulación en
   frecuencia

```

```

4
5 // -----
6 // lectura del archivo de entrada
7 // -----
8
9 [x_aux, Fm, bits] = wavread("Audio.wav");
10
11 size(x_aux)
12 Fm
13 bits
14
15 x_entrada = x_aux(1,:);
16 size(x_entrada)
17
18 // -----
19 // 1) Banda alta original -> desplazar a banda media
20 // -----
21
22 load("PasaAlto");
23
24 x_salida1 = convol(x_entrada, vhn1);
25 n = 0:(length(x_salida1)-1);
26
27 x_salidaPA = 2 * x_salida1 .* cos((1/6) * 2 * %pi * n);
28
29 load("PasaBandaFIR");
30 x_salida1C = convol(x_salidaPA, vhn);      // banda alta → banda
      media
31 wavwrite(x_salida1C, Fm, "intermedio.wav");
32
33 // -----
34 // 2) Banda media original -> desplazar a banda baja

```

```

35 // -----
36
37 x_salida2 = convol(x_entrada, vhn);
38 n2 = 0:(length(x_salida2)-1);
39
40 x_salidaPB = 2 * x_salida2 .* cos((1/6) * 2 * %pi * n2);
41
42 load("PasaBajosFIR");
43 x_salida2C = convol(x_salidaPB, vhn2);      // banda media → banda
      baja
44 wavwrite(x_salida2C, Fm, "intermedio2.wav");
45
46 // -----
47 // 3) Banda baja original -> desplazar a banda alta
48 // -----
49
50 x_salida3 = convol(x_entrada, vhn2);
51 n3 = 0:(length(x_salida3)-1);
52
53 x_salidaPL = 2 * x_salida3 .* cos((1/3) * 2 * %pi * n3);
54
55 load("PasaAlto");
56 x_salida3C = convol(x_salidaPL, vhn1);      // banda baja → banda alta
57 wavwrite(x_salida3C, Fm, "intermedio3.wav");
58
59 // -----
60 // combinación final de bandas codificadas
61 // -----
62
63 x_salida = x_salida1C + x_salida2C + x_salida3C;
64
65 // -----

```

```

66 // guardar salida codificada
67 // -----
68
69 wavwrite(x_salida, Fm, "salida.wav");

```

Decodificación del audio

```

1 // archivo leer_filtrar_con_filtro_FIR_y_guardar_audio.sce
2 //
3 // decodificación del audio codificado mediante filtros FIR
4 // invirtiendo el desplazamiento espectral aplicado en la
   codificación
5
6 clear;
7
8 // -----
9 // lectura del archivo codificado
10 // -----
11
12 [x_aux, Fm, bits] = wavread("salida.wav");
13
14 size(x_aux)
15 Fm
16 bits
17
18 x_entrada = x_aux(1,:);
19 size(x_entrada)
20
21 // quitar nivel DC
22 x_entrada = x_entrada - mean(x_entrada);
23
24 // -----
25 // 1) Banda alta codificada -> recuperar banda baja

```

```

26 // -----
27
28 load("PasaAlto");
29
30 x_salida1 = convol(x_entrada, vhn1);
31 n = 0:(length(x_salida1)-1);
32
33 // demodulación hacia bajas frecuencias (inverso de 1/3)
34 x_salidaPA = 2 * x_salida1 .* cos((2/3) * 2 * %pi * n);
35
36 load("PasaBajosFIR");
37 x_salida1C = convol(x_salidaPA, vhn2);      // banda baja recuperada
38 wavwrite(x_salidaPA, Fm, "intermedio.wav");
39
40 // -----
41 // 2) Banda baja codificada -> recuperar banda alta
42 // -----
43
44 load("PasaBandaFIR");
45 x_salida2 = convol(x_entrada, vhn);
46 n2 = 0:(length(x_salida2)-1);
47
48 // demodulación correspondiente (inverso de 1/6)
49 x_salidaPB = 2 * x_salida2 .* cos((1/6) * 2 * %pi * n2);
50
51 load("PasaAlto");
52 x_salida2C = convol(x_salidaPB, vhn1);      // banda alta recuperada
53 wavwrite(x_salida2, Fm, "intermedio2.wav");
54
55 // -----
56 // 3) Banda media codificada -> recuperar banda media
57 // -----

```

```

58
59 load("PasaBajosFIR");
60 x_salida3 = convol(x_entrada, vhn2);
61 n3 = 0:(length(x_salida3)-1);
62
63 // demodulación hacia zona media
64 x_salidaPL = 2 * x_salida3 .* cos((1/6) * 2 * %pi * n3);
65
66 load("PasaBandaFIR");
67 x_salida3C = convol(x_salidaPL, vhn); // banda media recuperada
68 wavwrite(x_salidaPL, Fm, "intermedio3.wav");
69
70 // -----
71 // combinación final de bandas decodificadas
72 // -----
73
74 x_salida = x_salida1C + x_salida2C + x_salida3C;
75
76 // -----
77 // guardar la señal decodificada
78 // -----
79
80 wavwrite(x_salida, Fm, "salidaDecodificada.wav");

```

Visualización del espectro

```

1 // archivo visualizar_el_espectro_de_secuencias_de_entrada_y_salida.
  sce
2 //
3 // grafica el espectro de diversas señales .wav mediante la TDF
4 // utilizando la implementación FFT de Scilab
5
6 clear;

```

```

7  close(winsid());
8
9  // -----
10 // 1) Espectro del audio original
11 // -----
12
13 [x_aux, Fm, bits] = wavread("Audio.wav");
14 size(x_aux), Fm, bits
15
16 x_entrada = x_aux(1,:); // usar una sola pista
17
18 K = 10;
19 x_n = [x_entrada, zeros(1,(K-1)*length(x_entrada))];
20 L = 2**floor(log2(length(x_n)));
21 x_n = x_n(1:L);
22
23 x_n_phi = fft(x_n);
24 v_phi = (1/L) * (0:L-1);
25
26 scf(0);
27 xgrid();
28 plot2d(Fm*v_phi, abs(x_n_phi), style=2)
29 xtitle("Espectro de la secuencia de entrada");
30
31
32 // -----
33 // 2) Espectro del audio codificado
34 // -----
35
36 [x_aux, Fm, bits] = wavread("salida.wav");
37 x_salida = x_aux(1,:);
38

```

```

39 x_n = [x_salida, zeros(1,(K-1)*length(x_salida))];
40 L = 2**floor(log2(length(x_n)));
41 x_n = x_n(1:L);
42
43 x_n_phi = fft(x_n);
44 v_phi = (1/L) * (0:L-1);
45
46 scf(1);
47 xgrid();
48 plot2d(Fm*v_phi, abs(x_n_phi), style=5)
49 xtitle("Espectro de la secuencia de salida codificada");
50
51
52 // -----
53 // 3) Espectro intermedio (intermedio.wav)
54 // -----
55
56 [x_aux, Fm, bits] = wavread("intermedio.wav");
57 x_inter = x_aux(1,:);
58
59 x_n = [x_inter, zeros(1,(K-1)*length(x_inter))];
60 L = 2**floor(log2(length(x_n)));
61 x_n = x_n(1:L);
62
63 x_n_phi = fft(x_n);
64 v_phi = (1/L) * (0:L-1);
65
66 scf(2);
67 xgrid();
68 plot2d(Fm*v_phi, abs(x_n_phi), style=5)
69 xtitle("Espectro intermedio");
70

```



```

71
72 // -----
73 // 4) Espectro intermedio 3 (intermedio3.wav)
74 // -----
75
76 [x_aux, Fm, bits] = wavread("intermedio3.wav");
77 x_inter = x_aux(1,:);
78
79 x_n = [x_inter, zeros(1,(K-1)*length(x_inter))];
80 L = 2**floor(log2(length(x_n)));
81 x_n = x_n(1:L);
82
83 x_n_phi = fft(x_n);
84 v_phi = (1/L) * (0:L-1);
85
86 scf(3);
87 xgrid();
88 plot2d(Fm*v_phi, abs(x_n_phi), style=5)
89 xtitle("Espectro intermedio 3");
90
91
92 // -----
93 // 5) Espectro intermedio 2 (intermedio2.wav)
94 // -----
95
96 [x_aux, Fm, bits] = wavread("intermedio2.wav");
97 x_inter = x_aux(1,:);
98
99 x_n = [x_inter, zeros(1,(K-1)*length(x_inter))];
100 L = 2**floor(log2(length(x_n)));
101 x_n = x_n(1:L);
102

```

```

103 x_n_phi = fft(x_n);
104 v_phi = (1/L) * (0:L-1);
105
106 scf(4);
107 xgrid();
108 plot2d(Fm*v_phi, abs(x_n_phi), style=5)
109 xtitle("Espectro intermedio 2");
110
111
112 // -----
113 // 6) Espectro del audio decodificado final
114 // -----
115
116 [x_aux, Fm, bits] = wavread("salidaDecodificada.wav");
117 x_entrada = x_aux(1,:);
118
119 x_n = [x_entrada, zeros(1,(K-1)*length(x_entrada))];
120 L = 2**floor(log2(length(x_n)));
121 x_n = x_n(1:L);
122
123 x_n_phi = fft(x_n);
124 v_phi = (1/L) * (0:L-1);
125
126 scf(5);
127 xgrid();
128 plot2d(Fm*v_phi, abs(x_n_phi), style=2)
129 xtitle("Espectro de la secuencia decodificada");

```